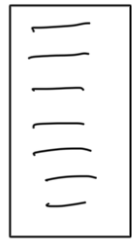


Process Threads Part 1

DAY 58
06/09/2025

Process $\rightarrow$ The Program Under Execution

Application/Program

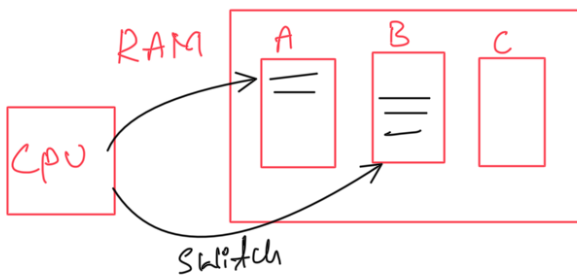


Process



Single process can spin off to multiple process

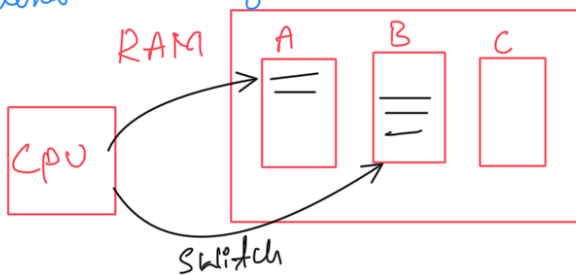
Application with Multiple process



CPU can execute one process at particular time, But When CPU switches from one process to another process there is some cost of CPU cycles are spent (There is some amount of Delay)

That Delay is called Context Switching

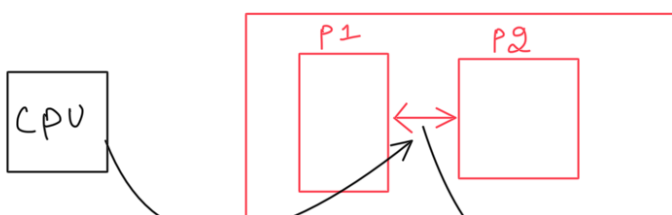
Context Switching



The more CPU perform Multiprocessing the more the Context Switching is happening

The more loading & unloading of Data occurs. the value storing in the registers increases

Two process will not communicate with each other directly



Inter process communication

1. Before Threads → Only Processes

Originally, OS scheduling was based purely on **processes**.

- A **process** = program in execution + its own memory + resources.
- Processes are **heavyweight**:
 - Creating a process (via `fork()` in Unix) takes time.
 - Context switching between processes involves switching memory space, registers, file descriptors, etc.

👉 Problem: For tasks that share a lot of common state (like memory, files, I/O buffers), **separate processes waste resources**.

2. Why Threads Came Into Picture

Threads were introduced to solve **efficiency** and **concurrency** problems.

1. Lightweight Execution Units

- A thread is a "mini-process" inside a process.
- Threads share the same memory and resources of the process but have their own execution stack and program counter.
- Much cheaper to create and switch than full processes.

2. Improved Concurrency

- Multiple threads in a process can run in **parallel** (on multicore CPUs) or **concurrently** (via time-slicing).
- Example: A web browser →
 - One thread handles UI
 - Another fetches data
 - Another plays audio

3. Better Resource Sharing

- Since threads in the same process share memory and files, communication is **easier and faster** (via shared memory).
- Inter-Process Communication (IPC) between separate processes is **much slower**.

3. Real-Life Example

✖ Imagine Microsoft Word:

- Without threads: You start saving a large file → whole app freezes until saving is done
- With threads:
 - One thread handles saving
 - One handles spelling check
 - One handles UI typing
 - Result: Smooth experience

4. Summary

Threads came into picture because:

- ✓ Processes were too heavyweight for fine-grained concurrency
- ✓ Needed responsiveness in interactive apps
- ✓ Shared memory/resources make communication easier
- ✓ Modern multicore CPUs demand parallelism

4. Responsiveness

- Applications remain responsive.
- Example: In a word processor →
 - One thread keeps UI interactive
 - Another thread saves file in background

5. Utilizing Multiprocessor Systems

- Modern CPUs are multicore. Threads allow true **parallel execution** of tasks.

Process vs Thread

Feature	Process	Thread
Definition	Independent program in execution with its own memory and resources.	Lightweight unit of execution inside a process.
Memory	Has its own address space (separate code, data, stack, heap).	Shares process's code, data, heap , but has its own stack & registers .
Creation cost	Expensive (requires allocation of new address space, PCB setup).	Cheaper (just create a new stack & thread control block).
Context switching	Heavy (OS switches memory mappings, registers, resources).	Light (only registers & stack pointer switch).
Communication	Inter-Process Communication (IPC) needed (pipes, sockets, shared memory).	Easy: direct read/write to shared memory of the process.
Isolation	Highly isolated: one process crash doesn't affect others (usually).	Less isolated: if one thread misbehaves (e.g., corrupts memory), entire process crashes.
Execution	Cannot run truly parallel within a single-core process (needs multiple processes).	Multiple threads of the same process can run in parallel on different CPU cores.
Examples	Chrome browser processes: one for each tab.	In each tab process, multiple threads: rendering, network, JavaScript execution.

What is Interprocess Communication (IPC)?

- Processes are **independent** → each has its own memory space.
- But many real-world tasks require **processes to work together and exchange data**.
- IPC = **mechanisms provided by the OS** to allow processes to:
 - Communicate (send/receive data)
 - Synchronize (coordinate execution order)

IPC

- Network protocols
- Messaging
- Shared Memory

Pros & Cons

✓ Process Pros

- Safer (crash in one process usually doesn't crash others)
- Better security & isolation.

✗ Process Cons

- Heavyweight.
- IPC is slow & complex.

✓ Thread Pros

- Lightweight, fast creation & context switching.
- Easier communication.
- Efficient for parallelism on multicore CPUs.

✗ Thread Cons

- One thread crash = entire process crash.
- Synchronization issues (race conditions, deadlocks).

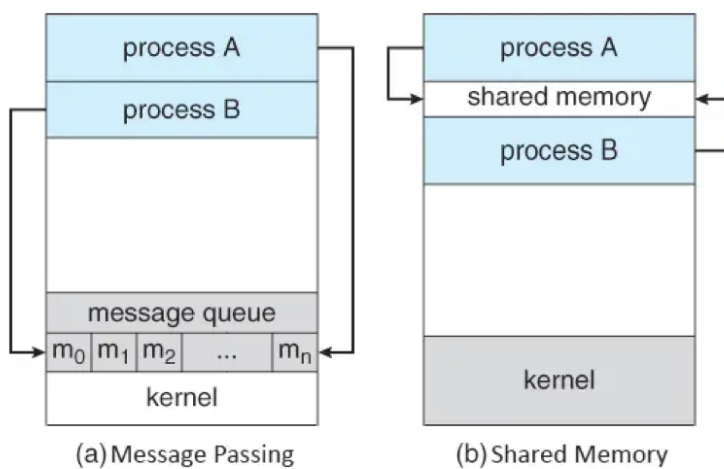
💡 Real-life example:

- YouTube app → One process. Inside it:
 - One thread plays video
 - One thread handles comments
 - One thread downloads recommendations
 - One thread responds to user taps

◆ Why do we need IPC?

1. **Data Sharing** → Example: Multiple applications use the same configuration.
2. **Computation Speedup** → Break large tasks into parallel processes.
3. **Modularity** → Separate processes for different components (e.g., browser tabs).
4. **Convenience** → Background processes (e.g., printer service) talk to apps.

Inter process Communication



But this cost some CPU cycle

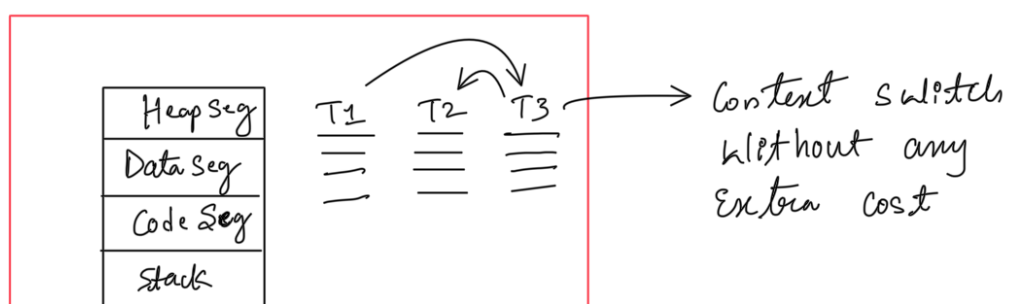
But the main reason why threads are introduced is

We want our application run parallelly but we don't want to pay CPU cost

To avoid extra overhead in inter process communication threads came into picture

Using same memory segment and threads the context switching is very low

Since threads use same memory inter process communication is very easy.



If process a Two un cooperative Neighbours
(like no Two process can communicate with each other without paying some cost)
or its very difficult to make Two process communicate

But The Threads are siblings living in the same House (Bcz siblings will share the same kitchen, same, wash room, same dining Hall)

So no Extra cost for context switching between the Two Threads Because of shared Memory.

1. When a thread locks itself from another thread?

This happens in synchronization using locks / mutexes:

- A lock (mutex) ensures only one thread enters a critical section (shared resource) at a time.
- If Thread A acquires the lock, Thread B must wait until Thread A releases it.

Example:

```
java  
  
synchronized void writeFile(String data) {  
    // Only one thread can be here at a time  
    file.write(data);  
}
```

- If Thread A is inside writeFile(), Thread B is blocked (locked out).
- If Thread A forgets to release the lock → deadlock may happen.

In short: Locking = blocking other threads from accessing shared resources until one finishes.

3. Why are Multithreaded Applications Hard to Debug?

Concurrency introduces non-determinism: execution order changes every time.

Challenges:

1. Race Conditions

- Two threads access shared data without synchronization → unpredictable results.
- Bug may appear only sometimes (hard to reproduce).

2. Deadlocks

- Thread A waits for lock1, Thread B waits for lock2, but both are holding the other lock → system halts.

3. Starvation

- A low-priority thread may never get CPU time if high-priority threads dominate.

4. Heisenbugs

- Debugging (adding logs, breakpoints) changes timing, making bug disappear.

5. Non-deterministic Execution

- The same code may behave differently in different runs (depending on CPU scheduling).

2. What is a Thread Pool?

A Thread Pool is a collection of pre-created worker threads that can be reused to execute tasks.

- Instead of creating a new thread for every task (expensive), tasks are submitted to a queue.
- Idle threads from the pool pick tasks and execute them.
- Once done, they go back to idle, ready for the next task.

Benefits:

- **Performance** → No overhead of creating/destroying threads repeatedly.
- **Control** → Limits the number of concurrent threads → avoids CPU overload.
- **Resource reuse** → Threads are recycled.

Real-world examples:

- Java's `ExecutorService` (`Executors.newFixedThreadPool(10)`).
- Web servers: Each client request is handled by a thread from a pool.

NOTE: In some situation
Threads also have
multiple access problem
Example: While writing

Summary

- Thread Locking → Prevents multiple threads from corrupting shared resources.
- Thread Pool → Pre-created threads that execute tasks efficiently, widely used in servers.
- Debugging Multithreaded Apps is Hard because of race conditions, deadlocks, starvation, and non-determinism.

Thread Safety

What is Thread Safety?

A piece of code is **thread-safe** if:

- It works correctly when multiple threads access it simultaneously.
- No data corruption, race conditions, or unexpected behavior occur.

👉 In other words: Thread-safe code = behaves the same in multi-threaded environment as in single-threaded.

How to Achieve Thread Safety

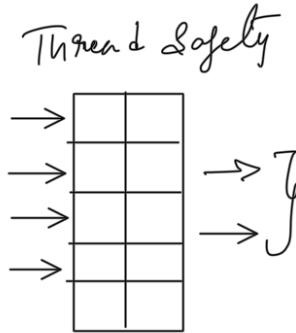
1. Synchronization (Locks / Mutexes)

- Use `synchronized` in Java or explicit locks.
- Ensures only one thread enters the critical section.

2. Atomic Operations

- Use classes like `AtomicInteger`, `AtomicBoolean`.
- Example:

```
java
AtomicInteger count = new AtomicInteger(0);
count.incrementAndGet(); // atomic, thread-safe
```



Example

Not Thread-Safe

```
java
class Counter {
    private int count = 0;

    public void increment() {
        count++; // not atomic → race condition possible
    }

    public int getCount() {
        return count;
    }
}
```

If two threads call `increment()` at the same time:

- Both read `count = 5`
- Both increment to 6
- Final value = 6 (instead of 7) → lost update.

Thread-Safe (with synchronization)

```
java
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++; // only one thread at a time
    }

    public synchronized int getCount() {
        return count;
    }
}
```

Now only **one** thread modifies count at a time → no race condition

3. Immutable Objects

- Immutable objects cannot be changed → inherently thread-safe.
- Example: `String` in Java.

4. Thread-local Storage

- Each thread has its own copy of a variable.
- Example: `ThreadLocal` in Java.

5. Concurrent Collections

- Instead of `HashMap` (not thread-safe), use `ConcurrentHashMap`.

Why Thread Safety Matters?

- Prevents data corruption in shared resources.
- Avoids race conditions and deadlocks.
- Ensures predictable, reliable behavior in multithreaded applications.

Real-Life Examples

- Banking system → Two users withdrawing from the same account at the same time.
- Web servers → Multiple requests accessing session data.
- Online shopping cart → Multiple threads updating stock levels.

✅ Thread Safety = code can be safely executed by multiple threads at the same time without breaking correctness.

What is the difference between process & threads?

Process has its own Memory Segments, its own Heap Segment, Data Segment, code Segment

Thread shares Memory Segment with its peer threads and it same has what the process is

In case of process content switching takes lot of time Bcz it take more CPU cycle to load the new content and run it

Where as in threads since it is in a same content, the content switching is easier.

Threads use same shared heap memory so inter process communication is easier, so they can directly access other peer threads and use it.